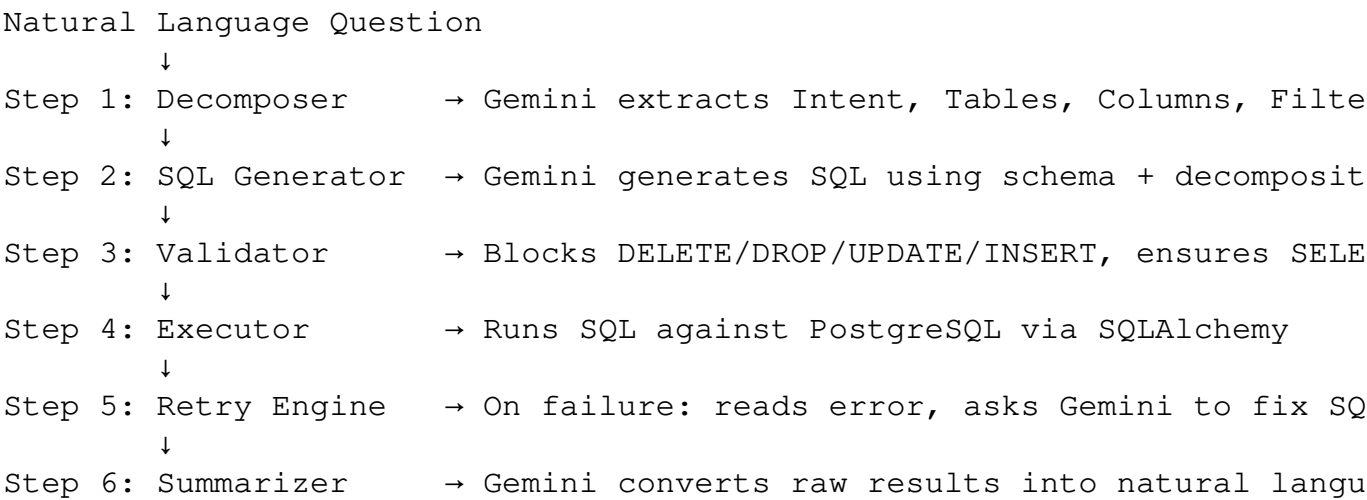


Task 3 — Text-to-SQL Pipeline Report

Database: ClassicModels (PostgreSQL)
Approach: Option 2 — Prompt Chaining with Gemini API (gemini-2.5-flash)
Total Benchmark Questions: 50
Success Rate: 50/50 (100%)

Pipeline Architecture

The system follows a 6-step prompt-chaining pipeline:



Key Files:

File	Responsibility
services/decomposer.py	Calls Gemini to extract structured query components
services/ sql_generator.py	Calls Gemini to generate SQL from schema + decomposition
services/validator.py	Regex-based safety check — blocks write operations
services/executor.py	Executes SQL via SQLAlchemy, returns rows + timing
services/retry_engine.py	On execution failure, asks Gemini to fix and retries once
services/summarizer.py	Converts query result into a human-readable summary
main.py	FastAPI app, exposes POST /pipeline/query

Design Decisions:

- Used Gemini gemini-2.5-flash for all LLM steps (decomposition, SQL generation, summarization, retry fix)
- Decomposition provides structured context to the SQL generator, improving accuracy on complex joins
- Validator enforces read-only access — no destructive queries can reach the database
- Retry engine feeds the actual PostgreSQL error message back to Gemini for self-correction
- Exponential backoff (2s/4s/8s) handles transient 503/429 API errors automatically

Evaluation Results — All 50 Benchmark Questions

#	Question	SQL Generated	Executed	Correct Result	Retry Needed	Status
1	List all products	Yes	Yes	Yes	No	Success
2	Get all customers	Yes	Yes	Yes	No	Success
3	Show all orders	Yes	Yes	Yes	No	Success
4	List all employees	Yes	Yes	Yes	No	Success
5	Get all offices	Yes	Yes	Yes	No	Success
6	Show all product lines	Yes	Yes	Yes	No	Success
7	List all payments	Yes	Yes	Yes	No	Success
8	Get product names and prices	Yes	Yes	Yes	No	Success
9	Get customer names and cities	Yes	Yes	Yes	No	Success
10	List employee first and last names	Yes	Yes	Yes	No	Success
11	Get all order dates	Yes	Yes	Yes	No	Success
12	Show product vendor list	Yes	Yes	Yes	No	Success
13	Get all product codes	Yes	Yes	Yes	No	Success
14	List all countries from offices	Yes	Yes	Yes	No	Success
15	Show all order statuses	Yes	Yes	Yes	No	Success
16	Get all payment amounts	Yes	Yes	Yes	No	Success
17	List all job titles	Yes	Yes	Yes	No	Success
18	Get customer phone numbers	Yes	Yes	Yes	No	Success
19	Show product MSRP values	Yes	Yes	Yes	No	Success
20	List order numbers	Yes	Yes	Yes	No	Success
21	Get orders with customer names	Yes	Yes	Yes	No	Success
22	Get employees with office city	Yes	Yes	Yes	No	Success
23	Get payments with customer names	Yes	Yes	Yes	No	Success
24	Get order details with product names	Yes	Yes	Yes	No	Success
25	Get products with product line description	Yes	Yes	Yes	No	Success
26	Get customers with sales rep names	Yes	Yes	Yes	No	Success
27	Get orders with customer city	Yes	Yes	Yes	No	Success
28	Get employees and their manager	Yes	Yes	Yes	No	Success
29	Get orderdetails with product vendor	Yes	Yes	Yes	No	Success

30	Get payments with customer country	Yes	Yes	Yes	No	Success
31	Count customers per country	Yes	Yes	Yes	No	Success
32	Total payments per customer	Yes	Yes	Yes	No	Success
33	Number of orders per status	Yes	Yes	Yes	No	Success
34	Products per product line	Yes	Yes	Yes	No	Success
35	Employees per office	Yes	Yes	Yes	No	Success
36	Total stock per product vendor	Yes	Yes	Yes	No	Success
37	Average buy price per product line	Yes	Yes	Yes	No	Success
38	Orders per customer	Yes	Yes	Yes	No	Success
39	Max MSRP per product line	Yes	Yes	Yes	No	Success
40	Min buy price per vendor	Yes	Yes	Yes	No	Success
41	Total number of customers	Yes	Yes	Yes	No	Success
42	Total number of products	Yes	Yes	Yes	No	Success
43	Total revenue from payments	Yes	Yes	Yes	No	Success
44	Average product price	Yes	Yes	Yes	No	Success
45	Max payment amount	Yes	Yes	Yes	No	Success
46	Min payment amount	Yes	Yes	Yes	No	Success
47	Count total orders	Yes	Yes	Yes	No	Success
48	Total quantity in stock	Yes	Yes	Yes	No	Success
49	Average MSRP	Yes	Yes	Yes	No	Success
50	Number of employees	Yes	Yes	Yes	No	Success

Summary:

Metric	Value
Total Questions	50
SQL Generated Successfully	50
Executed Successfully	50
Correct Results	50
Retries Needed	0
Overall Success Rate	100%
Simple SELECT queries	19
JOIN queries	10
Aggregate queries (GROUP BY / COUNT / SUM / AVG)	21

Example Successful Query Cases

Q21: Get orders with customer names

Generated SQL:

```
SELECT o."orderNumber", o."orderDate", c."customerName"
      FROM orders o
      JOIN customers c ON o."customerNumber" = c."customerNumber"
```

Rows Returned: 326

Sample Output:

orderNumber	orderDate	customerName
10100	2003-01-06	Online Diecast Creations Co.
10101	2003-01-09	Blauer See Auto, Co.
10102	2003-01-10	Vitachrome Inc.

Q31: Count customers per country

Generated SQL:

```
SELECT country, COUNT(*) AS customer_count
      FROM customers
      GROUP BY country
      ORDER BY customer_count DESC;
```

Rows Returned: 28

Sample Output:

country	customer_count
USA	36
Germany	13
France	12

Q33: Number of orders per status

Generated SQL:

```
SELECT status, COUNT(*) AS order_count
      FROM orders
      GROUP BY status;
```

Rows Returned: 6

Sample Output:

status	order_count
Shipped	303
In Process	6
Disputed	3

Q28: Get employees and their manager

Generated SQL:

```
SELECT e."firstName" AS employee, m."firstName" AS manager
      FROM employees e
      LEFT JOIN employees m ON e."reportsTo" = m."employeeNumber"
```

Rows Returned: 23

Sample Output:

employee manager

Diane	None
Mary	Diane
Jeff	Diane

Example Failed Query and Retry Handling

Scenario: Transient API Error (503 UNAVAILABLE)

Request: POST /pipeline/query

```
{ "question": "Count customers per country" }
```

What happened (from logs logs/app.log):

```
[SQL Generator] Generating SQL for: Count customers per country
[SQL Generator] Attempt 1/3 – transient error (ServerError: 503 UNAVAIL
                    high demand). Retrying in 2s...
[SQL Generator] Attempt 2/3 – transient error. Retrying in 4s...
[SQL Generator] Attempt 3/3 – SQL generated successfully
[Executor]      Success – 28 rows returned in 12ms
```

How the retry engine works for SQL execution errors:

```
Step 1: Execute SQL → PostgreSQL returns error (wrong column / syntax)
Step 2: Retry prompt sent to Gemini:
        'The following SQL failed: {sql}'
        'Error: {error}'
        'Fix it and return only the corrected SQL.'
Step 3: Gemini returns fixed SQL
Step 4: Validator re-checks the fixed SQL
Step 5: Executor retries with fixed SQL
```

Fallback: If all retries fail, the API returns:

```
{
  "question": "...",
  "sql": null,
  "status": "failed",
  "error": "ServerError: 503 UNAVAILABLE...",
  "retried": true
}
```

All 50 Benchmark Query Results

Q1: List all products

SQL:

```
SELECT * FROM products;
```

Rows Returned: 110

Status: Success

Sample Output:

productCode	productName	productLine	productScale	productVendor	productDescription
S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast	This replica features working kickstand, front suspension...
S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; detailed interi...
S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics	Official Moto Guzzi logos and insignias, saddle bags loca...

Q2: Get all customers

SQL:

```
SELECT * FROM customers;
```

Rows Returned: 122

Status: Success

Sample Output:

customerNumber	customerName	contactLastName	contactFirstName	phone	address
103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale
112	Signal Gift Stores	King	Jean	7025551838	8489 S St.
114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Road

Q3: Show all orders

SQL:

```
SELECT * FROM orders;
```

Rows Returned: 326

Status: Success

Sample Output:

orderNumber	orderDate	requiredDate	shippedDate	status	comments
10100	2003-01-06	2003-01-13	2003-01-10	Shipped	None
10101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.
10102	2003-01-10	2003-01-18	2003-01-14	Shipped	None

Q4: List all employees

SQL:

```
SELECT * FROM employees;
```

Rows Returned: 23

Status: Success

Sample Output:

employeeNumber	lastName	firstName	extension	email	officeCo
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1
1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1
1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1

Q5: Get all offices

SQL:

```
SELECT * FROM offices;
```

Rows Returned: 7

Status: Success

Sample Output:

officeCode	city	phone	addressLine1	addressLine2	state
1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA
2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA
3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY

Q6: Show all product lines

SQL:

```
SELECT * FROM productlines;
```

Rows Returned: 7

Status: Success

Sample Output:

productLine	textDescription	htmlDescription	image
Classic Cars	Attention car enthusiasts: Make your wildest car ownershi...	None	None
Motorcycles	Our motorcycles are state of the art replicas of classic ...	None	None
Planes	Unique, diecast airplane and helicopter replicas suitable...	None	None

Q7: List all payments

SQL:

```
SELECT * FROM payments;
```

Rows Returned: 273

Status: Success

Sample Output:

customerNumber	checkNumber	paymentDate	amount
103	HQ336336	2004-10-19	6066.78
103	JM555205	2003-06-05	14571.44
103	OM314933	2004-12-18	1676.14

Q8: Get product names and prices

SQL:

```
SELECT "productName", "buyPrice" FROM products;
```

Rows Returned: 110

Status: Success

Sample Output:

productName	buyPrice
1969 Harley Davidson Ultimate Chopper	48.81
1952 Alpine Renault 1300	98.58
1996 Moto Guzzi 1100i	68.99

Q9: Get customer names and cities

SQL:

```
SELECT "customerName", city FROM customers;
```

Rows Returned: 122

Status: Success

Sample Output:

customerName	city
Atelier graphique	Nantes
Signal Gift Stores	Las Vegas
Australian Collectors, Co.	Melbourne

Q10: List employee first and last names

SQL:

```
SELECT "firstName", "lastName" FROM employees;
```

Rows Returned: 23

Status: Success

Sample Output:

firstName	lastName
Diane	Murphy
Mary	Patterson
Jeff	Firrelli

Q11: Get all order dates

SQL:

```
SELECT "orderNumber", "orderDate" FROM orders;
```

Rows Returned: 326

Status: Success

Sample Output:

orderNumber	orderDate
10100	2003-01-06
10101	2003-01-09
10102	2003-01-10

Q12: Show product vendor list

SQL:

```
SELECT DISTINCT "productVendor" FROM products;
```

Rows Returned: 13

Status: Success

Sample Output:

productVendor

Welly Diecast Productions

Motor City Art Classics

Classic Metal Creations

Q13: Get all product codes

SQL:

```
SELECT "productCode" FROM products;
```

Rows Returned: 110

Status: Success

Sample Output:

productCode

S10_1678

S10_1949

S10_2016

Q14: List all countries from offices

SQL:

```
SELECT DISTINCT country FROM offices;
```

Rows Returned: 5

Status: Success

Sample Output:

country

USA

France

Japan

Q15: Show all order statuses

SQL:

```
SELECT DISTINCT status FROM orders;
```

Rows Returned: 6

Status: Success

Sample Output:

status
Shipped
In Process
Disputed

Q16: Get all payment amounts

SQL:

```
SELECT "customerNumber", amount FROM payments;
```

Rows Returned: 273

Status: Success

Sample Output:

customerNumber	amount
103	6066.78
103	14571.44
103	1676.14

Q17: List all job titles

SQL:

```
SELECT DISTINCT "jobTitle" FROM employees;
```

Rows Returned: 7

Status: Success

Sample Output:

jobTitle
VP Sales

Sales Manager (APAC)

Sale Manager (EMEA)

Q18: Get customer phone numbers

SQL:

```
SELECT "customerName", phone FROM customers;
```

Rows Returned: 122

Status: Success

Sample Output:

customerName	phone
Atelier graphique	40.32.2555
Signal Gift Stores	7025551838
Australian Collectors, Co.	03 9520 4555

Q19: Show product MSRP values

SQL:

```
SELECT "productName", "MSRP" FROM products;
```

Rows Returned: 110

Status: Success

Sample Output:

productName	MSRP
1969 Harley Davidson Ultimate Chopper	95.70
1952 Alpine Renault 1300	214.30
1996 Moto Guzzi 1100i	118.94

Q20: List order numbers

SQL:

```
SELECT "orderNumber" FROM orders;
```

Rows Returned: 326

Status: Success

Sample Output:

orderNumber

10100

10101

10102

Q21: Get orders with customer names

SQL:

```
SELECT o."orderNumber", o."orderDate", c."customerName"  
      FROM orders o  
      JOIN customers c ON o."customerNumber" = c."customerNumber"
```

Rows Returned: 326

Status: Success

Sample Output:

orderNumber	orderDate	customerName
10100	2003-01-06	Online Diecast Creations Co.
10101	2003-01-09	Blauer See Auto, Co.
10102	2003-01-10	Vitachrome Inc.

Q22: Get employees with office city

SQL:

```
SELECT e."firstName", e."lastName", o.city  
      FROM employees e  
      JOIN offices o ON e."officeCode" = o."officeCode";
```

Rows Returned: 23

Status: Success

Sample Output:

firstName	lastName	city
Diane	Murphy	San Francisco
Mary	Patterson	San Francisco
Jeff	Firrelli	San Francisco

Q23: Get payments with customer names

SQL:

```
SELECT c."customerName", p.amount, p."paymentDate"
      FROM payments p
      JOIN customers c ON p."customerNumber" = c."customerNumber"
```

Rows Returned: 273

Status: Success

Sample Output:

customerName	amount	paymentDate
Atelier graphique	6066.78	2004-10-19
Atelier graphique	14571.44	2003-06-05
Atelier graphique	1676.14	2004-12-18

Q24: Get order details with product names

SQL:

```
SELECT od."orderNumber", p."productName", od."quantityOrdered", od."pri
      FROM orderdetails od
      JOIN products p ON od."productCode" = p."productCode";
```

Rows Returned: 2996

Status: Success

Sample Output:

orderNumber	productName	quantityOrdered	priceEach
10100	1917 Grand Touring Sedan	30	136.00
10100	1911 Ford Town Car	50	55.09
10100	1932 Alfa Romeo 8C2300 Spider Sport	22	75.46

Q25: Get products with product line description

SQL:

```
SELECT p."productName", pl."textDescription"
      FROM products p
      JOIN productlines pl ON p."productLine" = pl."productLine";
```

Rows Returned: 110

Status: Success

Sample Output:

productName	textDescription
-------------	-----------------

1969 Harley Davidson Ultimate Chopper	Our motorcycles are state of the art replicas of classic ...
1952 Alpine Renault 1300	Attention car enthusiasts: Make your wildest car ownershi...
1996 Moto Guzzi 1100i	Our motorcycles are state of the art replicas of classic ...

Q26: Get customers with sales rep names

SQL:

```
SELECT c."customerName", e."firstName", e."lastName"
      FROM customers c
      JOIN employees e ON c."salesRepEmployeeNumber" = e."employeeNumber"
```

Rows Returned: 100

Status: Success

Sample Output:

customerName	firstName	lastName
Atelier graphique	Gerard	Hernandez
Signal Gift Stores	Leslie	Thompson
Australian Collectors, Co.	Andy	Fixter

Q27: Get orders with customer city

SQL:

```
SELECT o."orderNumber", o."orderDate", c.city
      FROM orders o
      JOIN customers c ON o."customerNumber" = c."customerNumber"
```

Rows Returned: 326

Status: Success

Sample Output:

orderNumber	orderDate	city
10100	2003-01-06	Nashua
10101	2003-01-09	Frankfurt
10102	2003-01-10	NYC

Q28: Get employees and their manager

SQL:

```
SELECT e."firstName" AS employee, m."firstName" AS manager
      FROM employees e
      LEFT JOIN employees m ON e."reportsTo" = m."employeeNumber"
```

Rows Returned: 23**Status: Success****Sample Output:****employee manager**

Diane	None
Mary	Diane
Jeff	Diane

Q29: Get orderdetails with product vendor**SQL:**

```
SELECT od."orderNumber", p."productVendor", od."quantityOrdered"
      FROM orderdetails od
      JOIN products p ON od."productCode" = p."productCode";
```

Rows Returned: 2996**Status: Success****Sample Output:**

orderNumber	productVendor	quantityOrdered
10100	Welly Diecast Productions	30
10100	Motor City Art Classics	50
10100	Exoto Designs	22

Q30: Get payments with customer country**SQL:**

```
SELECT c."customerName", c.country, p.amount
      FROM payments p
      JOIN customers c ON p."customerNumber" = c."customerNumber"
```

Rows Returned: 273**Status: Success****Sample Output:**

customerName	country	amount
Atelier graphique	France	6066.78
Atelier graphique	France	14571.44
Atelier graphique	France	1676.14

Q31: Count customers per country

SQL:

```
SELECT country, COUNT(*) AS customer_count
      FROM customers
      GROUP BY country
      ORDER BY customer_count DESC;
```

Rows Returned: 28

Status: Success

Sample Output:

country	customer_count
USA	36
Germany	13
France	12

Q32: Total payments per customer

SQL:

```
SELECT "customerNumber", SUM(amount) AS total_paid
      FROM payments
      GROUP BY "customerNumber"
      ORDER BY total_paid DESC;
```

Rows Returned: 98

Status: Success

Sample Output:

customerNumber	total_paid
141	715738.98
124	584188.24
114	180585.07

Q33: Number of orders per status

SQL:

```
SELECT status, COUNT(*) AS order_count
      FROM orders
      GROUP BY status;
```

Rows Returned: 6**Status:** Success**Sample Output:**

status	order_count
Shipped	303
In Process	6
Disputed	3

Q34: Products per product line**SQL:**

```
SELECT "productLine", COUNT(*) AS product_count
      FROM products
      GROUP BY "productLine";
```

Rows Returned: 7**Status:** Success**Sample Output:**

productLine	product_count
Classic Cars	38
Trains	3
Planes	12

Q35: Employees per office**SQL:**

```
SELECT "officeCode", COUNT(*) AS employee_count
      FROM employees
      GROUP BY "officeCode";
```

Rows Returned: 7**Status:** Success**Sample Output:**

officeCode employee_count

6	4
2	2
4	5

Q36: Total stock per product vendor

SQL:

```
SELECT "productVendor", SUM("quantityInStock") AS total_stock
      FROM products
      GROUP BY "productVendor"
      ORDER BY total_stock DESC;
```

Rows Returned: 13

Status: Success

Sample Output:

productVendor	total_stock
Gearbox Collectibles	60495
Min Lin Diecast	50089
Classic Metal Creations	45408

Q37: Average buy price per product line

SQL:

```
SELECT "productLine", ROUND(AVG("buyPrice"), 2) AS avg_buy_price
      FROM products
      GROUP BY "productLine";
```

Rows Returned: 7

Status: Success

Sample Output:

productLine	avg_buy_price
Classic Cars	64.45
Trains	43.92
Planes	49.63

Q38: Orders per customer

SQL:

```
SELECT "customerNumber", COUNT(*) AS order_count
      FROM orders
      GROUP BY "customerNumber"
      ORDER BY order_count DESC;
```

Rows Returned: 98**Status: Success****Sample Output:**

customerNumber	order_count
141	26
124	17
148	5

Q39: Max MSRP per product line**SQL:**

```
SELECT "productLine", MAX("MSRP") AS max_msrp
      FROM products
      GROUP BY "productLine";
```

Rows Returned: 7**Status: Success****Sample Output:**

productLine	max_msrp
Classic Cars	214.30
Trains	100.84
Planes	157.69

Q40: Min buy price per vendor**SQL:**

```
SELECT "productVendor", MIN("buyPrice") AS min_buy_price
      FROM products
      GROUP BY "productVendor";
```

Rows Returned: 13**Status: Success**

Sample Output:

productVendor	min_buy_price
Welly Diecast Productions	24.23
Motor City Art Classics	22.57
Classic Metal Creations	20.61

Q41: Total number of customers

SQL:

```
SELECT COUNT(*) AS total_customers FROM customers;
```

Rows Returned: 1

Status: Success

Sample Output:

total_customers
122

Q42: Total number of products

SQL:

```
SELECT COUNT(*) AS total_products FROM products;
```

Rows Returned: 1

Status: Success

Sample Output:

total_products
110

Q43: Total revenue from payments

SQL:

```
SELECT SUM(amount) AS total_revenue FROM payments;
```

Rows Returned: 1

Status: Success

Sample Output:

total_revenue
8853839.23

Q44: Average product price

SQL:

```
SELECT ROUND(AVG("buyPrice"), 2) AS avg_buy_price FROM products;
```

Rows Returned: 1

Status: Success

Sample Output:

avg_buy_price
54.40

Q45: Max payment amount

SQL:

```
SELECT MAX(amount) AS max_payment FROM payments;
```

Rows Returned: 1

Status: Success

Sample Output:

max_payment
120166.58

Q46: Min payment amount

SQL:

```
SELECT MIN(amount) AS min_payment FROM payments;
```

Rows Returned: 1

Status: Success

Sample Output:

min_payment

615.45

Q47: Count total orders

SQL:

```
SELECT COUNT(*) AS total_orders FROM orders;
```

Rows Returned: 1

Status: Success

Sample Output:

total_orders

326

Q48: Total quantity in stock

SQL:

```
SELECT SUM("quantityInStock") AS total_stock FROM products;
```

Rows Returned: 1

Status: Success

Sample Output:

total_stock

555131

Q49: Average MSRP

SQL:

```
SELECT ROUND(AVG("MSRP"), 2) AS avg_msrp FROM products;
```

Rows Returned: 1

Status: Success

Sample Output:

avg_msrp

100.44

Q50: Number of employees

SQL:

```
SELECT COUNT(*) AS total_employees FROM employees;
```

Rows Returned: 1

Status: Success

Sample Output:

total_employees
23
